micrograd

someonecantcode

February 4, 2026

Abstract

This is an explanation of my code, design choices, and how it's implementation is efficient. Additionally, we will also discuss potential future features and classes for new behaviors.

Contents

List of Theorems

Figure 1: hehehaha cool mega mind

1 Beginnings

A neural network is composed of layers which are composed of neurons. This structure (with a non-linear function) is very useful as it allows to approximate anything and shape it to any behavior we want. It's basically linear regression but on steroids. We will forward pass inputs and receive an output. With the output, we will calculate the loss and run back propagation for the individual gradients and apply gradient descent on the parameters to reduce the loss value. In turn, this will result in a model that is better suited toward the result. Repeating this process will eventually lead the loss function reaching a minimum.

Let us define all of the internal structures needed towards a Multi Layer Perceptron (MLP).

DEFINITION 1.1 (Neuron)**.** A neuron or perceptron is denoted by the following:

$$\mathrm{N} = \sum_{i=0}^{n} w_i x_i + b, \quad n \text{ being the input size.} \tag{1}$$

$$\mathrm{N} = \begin{bmatrix} x_0 & x_1 & \cdots & x_i \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_i \end{bmatrix} + \begin{bmatrix} b \end{bmatrix}, \quad i \text{ being the number of inputs.} \tag{2}$$

DEFINITION 1.2 (Layer)**.** A layer of neurons or a single layer perceptron is denoted by the following:

$$\mathrm{L} = \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_i \end{bmatrix}, \quad i \text{ being the amount of neurons.} \tag{3}$$

DEFINITION 1.3 (MLP)**.** A multi layer perceptron has multiple layers and it is denoted by the following:

$$\mathrm{MLP} = \begin{bmatrix} L_0 & L_1 & \cdots & L_i \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_a \end{bmatrix} & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_b \end{bmatrix} & \cdots & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_c \end{bmatrix} \end{bmatrix}, \quad a, b, c \text{ being the size of each layer.} \tag{4}$$

Now that we have some infrastructure to work with, we have to show how forward pass would work. Imagine we have a 2 layer (1 input/middle, 1 output). For the input layer we shall take in two inputs and output 4. The output layer must take 4 and we will let it output 1.

This is how the math would work out.

$$X_0 = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}, L_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}, N_i = \begin{bmatrix} x_0 & x_1 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} + \begin{bmatrix} b \end{bmatrix}$$

$$L_1 = \begin{bmatrix} N_0 \end{bmatrix}, N_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}^\top \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} N_0 & N_1 & N_2 & N_3 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} + \begin{bmatrix} b \end{bmatrix}$$

2 Back Propagation

To actually train and get the wanted behavior from a MLP, we must obtain the direction of largest growth or smallest otherwise known as the gradient of the parameters. With the gradient, we can use that to move in a direction of lower loss. With a little bit of multi-variable calculus and chain rule, we can obtain all of the local derivatives and in turn, their gradients. Here is the math behind it.

The most rudimentary case is to imagine taking derivatives at the root.

$$\frac{\partial L}{\partial c} = \frac{\partial L}{\cancel{\partial L}}^{1} \frac{\partial L}{\partial c} = \frac{\partial L}{\partial c} \tag{5}$$

However, imagine if we have a function of two variables with one being a composite. In this example, it should become clear that we will always calculate the outer derivative and multiply it by the inner derivative. At the start, we will simply take it's derivative and multiply it by one.

DEFINITION 2.1 (Chain Rule)**.** With that, back propagation is defined by the chain rule.

$$L(c, d) = c + d, \quad c(a) = a + b; \tag{6}$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}\frac{\partial c}{\partial a}} \tag{7}$$

Acculumating gradients:

$$L(c, d) = c(a) + d(a), \quad c(a) = a + b,\ d(a) = a + c \tag{8}$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}\frac{\partial c}{\partial a} + \frac{\partial L}{\partial d}\frac{\partial d}{\partial a}} \tag{9}$$

DEFINITION 2.2 (Gradient)**.** We can take this farther and define the gradient of the loss function.

$$\boxed{\nabla L = \left\langle \frac{\partial L}{\partial a_0}, \dots \frac{\partial L}{\partial a_i} \right\rangle} \tag{10}$$

This is a very powerful tool and the next thing to tackle is how we order our function so that we can create an algorithm to start from the root and propagate back.

2.1 Topological Sort

With back propagation, we will need to be able to know what order to propagate. This is basically a job scheduling problem so topological sort will be the perfect algorithm for this. Topological sort works by applying a post-order DFS and reversing the list. This can easily be done using a recursive formula to completely search each branch. Reversing should be extremely simple with a library.

Algorithm 1 Topological Sort

Require: $n \geq 0$
Ensure: $y = x^n$
  $y \leftarrow 1$
  $X \leftarrow x$
  $N \leftarrow n$
  while $N \neq 0$ **do**
    if N is even **then**
      $X \leftarrow X \times X$
      $N \leftarrow \frac{N}{2}$ ▷ This is a comment
    else if N is odd **then**
      $y \leftarrow y \times X$
      $N \leftarrow N - 1$
    end if
  end while

Algorithm 2 DFS for Topological Sort

```
procedure DFS(v, topo, visited)
    if v ∉ visited then
        visited ← visited ∪ {v}
        for all child ∈ v.prev_children do
            DFS(child, topo, visited)
        end for
        append(topo, v)
    end if
end procedure
```